

Android Cursor 全面详解：从概念到实战

在 Android 数据操作中，Cursor（游标）是连接应用与系统数据（如数据库、ContentProvider 提供的媒体文件/联系人等）的核心桥梁。无论是查询 SQLite 数据库，还是通过 MediaStore 读取图片、通过 ContactsContract 访问联系人，都离不开 Cursor 的使用。本文将从本质、使用流程、核心方法、生命周期管理及实战案例等维度，彻底讲透 Cursor。

一、Cursor 本质与核心作用

1. 什么是 Cursor？

Cursor 是 Android 提供的一个**数据查询结果集的抽象接口**，它类似于数据库中的“游标”概念——指向查询结果集的当前行，允许开发者逐行遍历数据、获取指定列的值，并能判断结果集的状态（如有无数据、是否到末尾等）。

简单来说：当你通过 `SQLiteDatabase.query()` 或 `ContentResolver.query()` 发起数据查询时，系统不会直接返回所有数据，而是返回一个 Cursor 对象。通过这个“游标”，你可以按需获取数据，避免一次性加载大量数据导致内存溢出。

2. 核心作用

- **遍历结果集**：通过移动游标位置，逐行访问查询结果；
- **获取数据**：根据列名或列索引，获取当前行指定列的数据（支持字符串、整数、长整数、字节数组等多种类型）；
- **判断状态**：判断结果集是否为空、游标是否移动到开头/末尾等；
- **获取元数据**：获取结果集的列名、列数等信息。

3. 适用场景

Cursor 主要用于“查询操作”，常见场景包括：

- 操作 SQLite 数据库（如查询用户信息表）；
- 通过 ContentProvider 访问系统数据（如 MediaStore 图片、ContactsContract 联系人、Telephony.Sms 短信）；
- 第三方 ContentProvider 数据查询。

二、Cursor 核心属性与状态

Cursor 的核心是“位置”，理解其位置状态是正确使用的基础。Cursor 的位置从 -1 开始（初始位置，指向结果集之前），随着移动方法的调用，位置会在结果集的行之间变化。

1. 关键状态

状态	说明	对应方法
初始状态	游标指向结果集第一行之前，位置为 -1	Cursor 刚创建时的默认状态
有效行	游标指向结果集中的某一行（位置 ≥ 0 ）	<code>moveToFirst()</code> 、 <code>moveToNext()</code> 成功后
结果集末尾	游标指向结果集最后一行之后，位置 $\geq$ 结果集行数	<code>moveToNext()</code> 返回 false 时
空结果集	查询无数据，游标始终在初始位置（-1）	<code>getCount()</code> 返回 0

2. 核心属性方法

- `getCount()`：返回结果集的总行数（重要！判断是否有数据的核心方法）；
- `getPosition()`：返回当前游标的位置（初始为 -1）；
- `isBeforeFirst()`：判断游标是否在结果集之前（初始状态）；
- `isAfterLast()`：判断游标是否在结果集之后（遍历结束）；
- `isNull(int columnIndex)`：判断当前行指定列的值是否为 null。

三、Cursor 核心使用流程

Cursor 的使用遵循“创建 → 遍历 → 获取数据 → 关闭”的固定流程，任何环节的疏漏都可能导致内存泄漏或异常。

1. 完整使用步骤

- 获取 Cursor**：通过查询方法（如 `query()`）获取 Cursor 对象；
- 判断有效性**：通过 `getCount()` 或 `moveToFirst()` 判断结果集是否有数据；
- 遍历结果集**：通过移动游标方法（如 `moveToNext()`）逐行遍历；
- 获取数据**：在遍历中通过类型方法（如 `getString()`）获取当前行数据；
- 关闭 Cursor**：遍历结束后必须关闭 Cursor，释放资源。

2. 核心移动方法

游标移动是遍历数据的关键，常用移动方法返回 boolean 值，表示移动是否成功。

方法	作用	适用场景
<code>moveToFirst()</code>	移动到第一行，成功返回 true（无数据返回 false）	首次遍历前，判断是否有数据
<code>moveToNext()</code>	移动到下一行，成功返回 true（到末尾返回 false）	循环遍历所有数据（最常用）
<code>moveToLast()</code>	移动到最后一行，成功返回 true	需要获取最后一条数据时
<code>moveToPosition(int position)</code>	移动到指定位置（position 从 0 开始）	定位到特定行数据
<code>move(int offset)</code>	相对当前位置移动 offset 行（正数向后，负数向前）	需要相对位置移动时

3. 数据获取方法

获取当前行数据时，需通过“列索引”或“列名”定位列，推荐先通过列名获取索引（避免硬编码索引，提高可读性）。

核心类型方法（根据列的数据类型选择）：

- `getString(int columnIndex)`：获取字符串类型数据（如联系人姓名、图片名称）；
- `getInt(int columnIndex)`：获取整数类型数据（如 ID、年龄）；
- `getLong(int columnIndex)`：获取长整数类型数据（如时间戳、文件大小）；
- `getBlob(int columnIndex)`：获取字节数组（如图片二进制数据）；
- `getDouble(int columnIndex)`：获取浮点型数据（如经纬度）。

通过列名获取索引的方法：`getColumnIndex(String columnName)` 或 `getColumnIndexOrThrow(String columnName)`（后者在列名不存在时抛出异常，推荐用于确定的列名）。

四、Cursor 生命周期与资源管理（重中之重）

Cursor 是“重量级资源”，关联数据库连接或 ContentProvider 资源，若不及时关闭会导致内存泄漏、数据库连接耗尽等严重问题。

1. 必须关闭 Cursor 的场景

- 遍历完成后（无论结果集是否有数据）；
- Activity/Fragment 销毁时（避免组件销毁后 Cursor 仍占用资源）；
- 查询失败或发生异常时（需在 finally 块中关闭）。

2. 安全关闭方式

推荐使用“try-catch-finally”结构，确保 Cursor 无论是否发生异常都能被关闭。

Code block

```
1  Cursor cursor = null;
2  try {
3      // 1. 获取 Cursor (示例: 查询SQLite数据库)
4      cursor = db.query("user", null, null, null, null, null, null);
5
6      // 2. 判断并遍历数据
7      if (cursor != null && cursor.moveToFirst()) {
8          do {
9              // 通过列名获取索引
10             int idIndex = cursor.getColumnIndexOrThrow("id");
11             int nameIndex = cursor.getColumnIndexOrThrow("name");
12
13             // 获取数据
14             int id = cursor.getInt(idIndex);
15             String name = cursor.getString(nameIndex);
16
17             // 处理数据 (如添加到列表)
18             } while (cursor.moveToNext()); // 循环移动到下一行
19     }
20 } catch (Exception e) {
21     e.printStackTrace();
22 } finally {
23     // 3. 关闭 Cursor (核心!)
24     if (cursor != null && !cursor.isClosed()) {
25         cursor.close();
26     }
27 }
```

3. 自动化管理：CursorLoader（推荐）

手动管理 Cursor 容易遗漏关闭操作，Android 提供 `CursorLoader`（属于 Jetpack 组件），可自动管理 Cursor 的生命周期：

- 自动在后台线程执行查询（避免主线程阻塞）；
- 组件销毁时自动关闭 Cursor；
- 数据变化时自动重新查询并更新 Cursor。

使用 CursorLoader 需实现 `LoaderManager.LoaderCallbacks<Cursor>` 接口，核心是 `onLoadFinished()` 方法接收 Cursor 并使用。

五、实战案例：结合 ContentProvider 使用 Cursor

通过 MediaStore 查询图片是 Cursor 的典型应用场景，以下是完整示例（兼容 Android 10+）。

1. 权限配置（AndroidManifest.xml）

Code block

```
1  <!-- Android 9及以下需要存储权限 -->
2  <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"
    android:maxSdkVersion="28" />
3  <!-- Android 13+ 需要图片权限 -->
4  <uses-permission android:name="android.permission.READ_MEDIA_IMAGES" />
```

2. 核心代码（通过 MediaStore 查询图片）

（注：文档部分内容可能由 AI 生成）